What is a Cookie

farnoodReact | farnoodlotfali

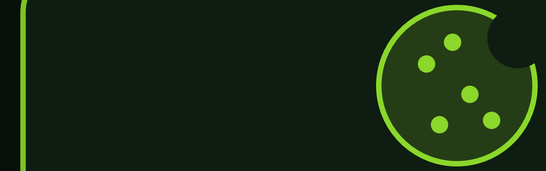
What is a Cookie?

> A cookie is a piece of data that is stored on your computer to be accessed by your browser.
>
> Cookies are saved as key/value pairs.

For example, you can create a cookie named username:

cookie.js

```
document.cookie = "username=John";
```



KEY username

VALUE John

Stored in the browser as a key/value pair

Why Do You Need a Cookie?

Cookies are used to remember information about the user profile (such as username). It basically involves two steps:

1 First visit

When a user visits a web page, the user profile can be stored in a cookie.

```
document.cookie = "username=John";
```

2 Next visit

Next time the user visits the page, the cookie remembers the user profile.

```
console.log(document.cookie);
```

Cookie Options

There are a few options available for a cookie



expires

By default, a cookie is a session cookie and is generally removed when the browser session ends. Set an expiry date (in UTC time) to keep it longer.

```
const d = new Date();
d.setDate(d.getDate() + 7);
document.cookie =
  `username=John; expires=${d.toUTCString()}`;
```



path

By default, a cookie's path is derived from the URL path of the page that sets it. You can specify a different path using the path parameter.

```
document.cookie =
  "username=John; path=/services";
```

More Cookie Options

Two more options control how long a cookie lives and where it is sent



Max-Age

Sets how many seconds the cookie lives from now. Max-Age=0 deletes it. If both Expires and Max-Age are set, Max-Age wins.

```
// Keep the cookie for 7 days
document.cookie =
  "username=John; Max-Age=604800";
```



Domain

Makes the cookie available to that domain and all its subdomains. If omitted, it is sent only to the exact host that set it.

```
// Share across subdomains
document.cookie =
  "username=John; Domain=example.com";
```

Security Options

Three attributes that protect cookies, especially authentication cookies



Secure

Sends the cookie only over HTTPS. Browsers make an exception for localhost.

```
"username=John; Secure"
```



HttpOnly

Hides the cookie from JavaScript, so document.cookie can't read it. Only the server can set it.

```
Set-Cookie: id=1; HttpOnly
```



SameSite

Controls if the cookie is sent on cross-site requests (helps against CSRF). Values: Strict, Lax, None.

```
"username=John; SameSite=Lax"
```

Example: Create a Cookie

Cookies are saved as key/value pairs. Create one, read it back, then add the expires and path options.

create.js

```
// Create a cookie (key/value pair)
document.cookie = "username=John";

// Read all cookies
console.log(document.cookie);
// "username=John"
```

options.js

```
// Keep it for 7 days (expires is in UTC)
const d = new Date();
d.setDate(d.getDate() + 7);
document.cookie =
  `username=John; expires=${d.toUTCString()}`;

// Limit it to a path
document.cookie = "username=John; path=/services";
```

expires keeps the cookie after the browser closes, and path sets where it applies.

Example: Delete a Cookie

Delete a cookie by setting its expiry date to a date in the past. No cookie value is needed.

delete.js

```
// Delete a cookie: set a past expiry date
document.cookie =
  "username=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/";

// Or expire it immediately with Max-Age
document.cookie = "username=; Max-Age=0; path=/";

// path (and domain) must match the original cookie
```

Deleting checklist

- Use a past expiry date (or Max-Age=0)
- No need to specify a cookie value
- Use the same path as the original cookie

If the path (or domain) doesn't match the original cookie, the browser won't delete it.

path must match the cookie you created, so the right one gets deleted.

Putting It Together

Combine options in one cookie. For login cookies, set them from the server so you can add HttpOnly.

client.js

```
// From JavaScript (HttpOnly is not possible here)
document.cookie =
  "theme=dark; Max-Age=604800; Path=/; Secure; SameSite=Lax";
```

response header

```
// From the server (best for login cookies)
Set-Cookie: sessionId=abc123;
  Max-Age=3600; Path=/;
  Secure; HttpOnly; SameSite=Strict
```

HttpOnly is server-only, so JavaScript can never read that cookie.

Cookie Options at a Glance

Option	Default Behavior	How to Change It
Lifetime	Session cookie; removed at session end	`expires=<UTC date> / Max-Age=<sec>`
Scope	Path from URL; host-only domain	`path=/services; Domain=example.com`
Security	Sent over HTTP; readable by JS	`Secure; HttpOnly; SameSite=Lax`
Format	Saved as a key/value pair	`document.cookie = "username=John"`
Delete	Stays until it expires	**Past expiry (or Max-Age=0) + same path**

Use expires or Max-Age to keep it, path and Domain to scope it, and the security flags to protect it.

Session cookies are generally removed when the browser session ends; behavior may vary by browser.

Key Takeaways

1. **Key/value data**
   A cookie is a piece of data stored on your computer, saved as key/value pairs.

2. **Remembers the user**
   Cookies remember profile info, such as a username, the next time the user visits.

3. **Lifetime: expires / Max-Age**
   By default it is a session cookie. Set an expiry date or Max-Age to keep it longer.

4. **Scope: path / Domain**
   The path comes from the page's URL by default. Domain shares it with subdomains.

5. **Security: Secure, HttpOnly, SameSite**
   HTTPS only, hidden from JavaScript, and limited on cross-site requests.

6. **Deleting a cookie**
   Use a past expiry date (or Max-Age=0) with the same path as the original cookie.